Deliverable 2: Architecture Document

NABAT-AI: Khaleeji Nabati Poetry Digitization & RAG System

Student: Asma Salem Mubarak Najem Aljneibi

Course: MAAI1704 - Generative AI

Date: April 2026

Abstract

This architecture document presents a literature-grounded Hybrid Deterministic-to-Generative pipeline for the NABAT-AI project, informed by a comprehensive review of 67 academic and technical sources spanning Arabic HTR, Vision-Language Models, and state-of-the-art Retrieval-Augmented Generation techniques, and validated against direct visual inspection of the 23 target manuscripts.

The system employs two specialized workers: Al-Nassikh (The Scribe), a rigid deterministic ETL pipeline that converts handwritten Khaleeji Nabati manuscript images into gold-standard multi-variant transcriptions with zero hallucination guarantee, with a dedicated pre-processing stack specifically engineered for the six visual challenges observed in the actual manuscripts; and Fatat Al Arab Agent, a LangGraph-orchestrated Advanced RAG agent using HyDE query translation, ColBERT token-level embeddings, Multi-Representation Indexing, CRAG, and Self-RAG. Every architectural decision is grounded in both literature findings and direct manuscript evidence.

Table of Contents

Section 1: Problem Recap and System Goals

1.1 Problem Statement

Khaleeji Nabati poetry represents a centuries-old oral and written tradition of the Arabian Gulf, recognized by UNESCO as intangible cultural heritage. Despite its immense cultural value, this legacy faces a severe digitization bottleneck. As highlighted by Dr. Sultan Al Ameemi, the currently accessible digital texts represent merely 10-20% of the entire Nabati Poetry heritage. The vast majority remains locked away in physical archives, largely due to the extreme technical difficulty of accurately recognizing and digitizing complex, dialect-specific handwriting.

The core challenge is to digitize these manuscripts with heritage-grade accuracy and expose them through a dialect-aware semantic search system that scholars can query in natural language. Key technical barriers include: (1) highly variable handwriting across centuries and regional scribes with dual linguistic dimensions (al-Mantuq and al-Maktub); (2) complex Arabic script with diacritical marks critical to poetic interpretation; (3) Khaleeji dialectal variation requiring domain-specific language understanding; (4) no existing dialect atlas or phonological rule set for Nabati poetry; (5) scarce training data for specialized HTR in dialectal poetry contexts; and (6) severe physical degradation of manuscript pages as evidenced by direct visual inspection of the target corpus.

Addressing this critical gap, the NABAT-AI project aims to pioneer a scalable solution by digitizing an initial corpus of 50 pages from historical archives (e.g., the Sowayan collection consisting of 782+ pages). The objective is to transform these manuscripts into a fully searchable, text-queryable Retrieval-Augmented Generation (RAG) system that navigates complex Khaleeji dialects without LLM hallucination.

1.2 Target Users

Cultural Institutions: Poetry Academy, Abu Dhabi Heritage Authority, Abu Dhabi Arabic Language Centre, National Archives.

Poets, Writers, and Academic Researchers: Complete datasets of digitized Khaleeji poems for linguistic, literary, and historical analysis.

Students and Educators: Nabati poetry in classroom curricula, cultural studies, and language learning.

General Public: Enthusiasts and poetry lovers with genealogical or cultural interest in Nabati traditions.

1.3 Desired System Outcomes

Expert-Validated Digitization: Convert handwritten manuscripts into verified, machine-readable transcriptions. Each verse includes three layers: al-Maktub (original), orthographic (modernized), and al-Mantuq (phonetic).

Traceable Poetic Indexing: Create a dialect-aware knowledge base where every entry is linked to its physical source, enabling scholars to navigate by poet, manuscript ID, or verse pattern.

Grounded RAG Retrieval: Deliver relevant poetic passages in response to dialectal queries, supported by mandatory inline citations and direct links to manuscript folio images to eliminate hallucination.

Linguistic Heritage Integrity: Preserve archaic Gulf spellings and dialectal nuances exactly as written, bypassing standard MSA auto-correction to maintain historical authenticity (WER < 10%).

1.4 Key Success Metrics

These might evolve as implementation progresses

Metric	Target	Rationale
End-to-End Throughput	50 folios	Establishes the "Gold Standard" baseline within the semester constraint.
Citation Precision	> 95%	Ensures every response is grounded in a verified manuscript source (Reliability).
Word Error Rate (WER)	< 10%	Maintains high-fidelity transcription for archaic dialectal forms.
Response Relevance	> 85%	3-point qualitative assessment (Relevant/Partial/Irrelevant) vs. expert judgment.

1.5 Why an Agentic Approach

A monolithic pipeline cannot express the conditional complexity of this problem. The system must decide when to auto-accept transcription, when to invoke a jury of models, when to escalate to human experts, and how to gracefully degrade. A hybrid agentic approach provides: (1) Deterministic guarantees where Worker 1 uses no LLM creativity during transcription; (2) Intelligent orchestration where Worker 2 uses a LangGraph agent to synthesize grounded responses; (3) Confidence-based routing with explicit thresholds; and (4) Resilience where failure in one component triggers fallbacks rather than cascading failure.

Section 1.6: Literature Review and Architectural Grounding

This section synthesizes findings from a comprehensive literature review of 67 academic and technical sources to ground every architectural decision in the NABAT-AI system. The review spans three domains:

(1) Arabic Handwritten Text Recognition models and platforms,
(2) document layout analysis for historical manuscripts, and
(3) state-of-the-art Retrieval-Augmented Generation techniques.

1.6.1 Arabic HTR: Model Architectures and Performance

The literature identifies three generations of neural architectures for Arabic manuscript recognition. The first generation CRNN-CTC architecture (ResNet + Bi-LSTM + CTC) eliminated explicit character segmentation but degrades significantly on historical manuscripts [1, 4, 6, 12, 27]. The second generation introduced Vision Transformer (ViT) encoders paired with RoBERTa decoders, achieving substantial gains on historical Arabic data: HATFormer reaches 8.6% CER on Muharaf (51% improvement over CRNN baselines) [2, 28, 29], while Qalam — a Vision Encoder-Decoder (VED) model built on SwinV2 + RoBERTa — achieves a state-of-the-art WER of 0.80% on handwriting recognition after training on over 4.5 million Arabic manuscript images, with explicit architectural focus on diacritics (tashkeel and nuqat) recognition (ArabicNLP 2024) [27]. Crucially, Qalam uses SwinV2 rather than a plain ViT: SwinV2 processes high-resolution manuscript images by dividing them into hierarchical local patches, preserving the fine spatial context needed for diacritical mark detection at 300 DPI. The third generation VLMs (QARI-OCR on Qwen2-VL, Katib-OCR on Qwen3.5) achieve sub-5% CER while adding end-to-end structural parsing capabilities [5, 7, 33, 34, 35].

Architecture	Examples	Core Mechanism	Best Use	Limitation
CRNN-CTC	DeepAHR, Tesseract LSTM	ResNet + Bi-LSTM + CTC	Clean printed text	Fails on complex historical layouts
Gen 2 Transformer (VED)	Qalam (SwinV2+RoBERTa)	Hierarchical patch encoder + cross-attention RoBERTa decoder	High-res manuscripts, diacritics precision	Computationally heavy; needs large training data
Gen 2 Transformer (ViT)	HATFormer, TrOCR	ViT encoder + RoBERTa decoder	Historical manuscripts, line-level HTR	Requires precise fine-tuning per domain
Gen 3 VLMs	QARI-OCR, Katib-OCR	ViT + autoregressive LLM	Zero-shot, page-level structure	May hallucinate without bounding boxes

Architectural grounding: Al-Nassikh uses Qalam (Gen 2 VED — SwinV2+RoBERTa, WER 0.80%, 4.5M+ training images, diacritics specialist), QARI-OCR v0.3 (Gen 3 VLM — structural HTML awareness on Qwen2-VL), and Arabic-Nougat (Gen 3 Hybrid — Markdown document output) as ensemble members, with HATFormer (Gen 2 ViT+RoBERTa) as jury tiebreaker — each generation addressing different failure modes identified in the literature.

1.6.2 Layout Analysis for Arabic Poetry Manuscripts

Arabic poetry's Sadr-Ajuz two-column system causes standard OCR engines to produce interleaved text [17, 18, 22]. The literature recommends Kraken v5 BLLA segmentation for Arabic manuscripts, validated by the KITAB project at scale [40, 41, 44]. PyArud provides deterministic Arabic poetic meter analysis for post-OCR validation [23, 24, 26].

Architectural grounding: Al-Nassikh uses Kraken v5 BLLA with a dedicated layout classifier to pre-identify page zones before segmentation. PyArud meter validation is integrated as a confidence signal in the routing step.

1.6.3 Platforms: eScriptorium, Transkribus, and Specialized Tools

As per the literature, eScriptorium/Kraken provides maximum control (>97% accuracy) and free academic use but requires technical expertise. Transkribus offers 300+ models but requires per-page payment and 5,000-15,000 lines of training data [47, 48]. Calfa Vision specializes in Maghrebi Arabic. Zinki offers a simplified workflow [57]. eScriptorium was selected for our HITL platform based on its iterative correction workflow and Kraken integration [44, 45, 46].

1.6.4 State-of-the-Art RAG Techniques

Based on a comprehensive review of RAG architectures, eight techniques were selected for Fatat Al Arab Agent:

Query Translation

HyDE (Hypothetical Document Embeddings): Generates a hypothetical Nabati verse from the query and uses its embedding for retrieval — bridging the gap between modern Arabic questions and archaic poetic embedding space.

Multi-Query / RAG-Fusion (Bilingual): Accepts queries in Arabic OR English. Language detection routes English queries through an Arabic translation step first. Multi-query expansion generates 3-5 variants in BOTH languages simultaneously — e.g., an English query about 'longing' produces Arabic variants غياب، اشتياق، حنين alongside English rephrases. All variants are merged via RRF, ensuring dialectal Khaleeji metaphors are retrieved even when the user queries in English.

Step-Back Prompting: Abstracts specific verse questions to broader poetic themes, retrieving relevant context first.

Routing and Query Construction

Logical Routing: LLM function calling routes quantitative metadata queries to PostgreSQL and thematic queries to Qdrant vector search.

Self-Query Retriever: Extracts structured metadata constraints (poet, manuscript ID, theme) as hard pre-filters before vector search.

Indexing

Semantic Splitter: Splits text at verse/stanza boundaries using PyArud metrical analysis — never mid-verse.

Multi-Representation Indexing: Embeds MSA prose summaries for retrieval but returns original Khaleeji text — bridging the vocabulary gap without corrupting heritage data.

ColBERT Token-Level Embeddings: Per-token late interaction matching preserves nuance of specific dialectal words that single dense vectors compress away.

Retrieval and Generation

CRAG (Corrective RAG): LLM grades retrieved documents for relevance; triggers query re-writing if all documents fail. Prevents irrelevant passages contaminating synthesis.

Self-RAG (Retrieve, Read, Reflect): Post-synthesis self-evaluation for faithfulness, relevance, completeness — critical for catching hallucinated archaic Khaleeji word definitions. Retries up to 2x before flagging for expert review.

1.6.5 Datasets and Evaluation Benchmarks

Key benchmarks: Muharaf (NeurIPS 2024, 1,600+ pages) is the primary Arabic HTR benchmark [63, 64]. TariMa achieves 97.21% accuracy with Kraken on Maghrebi scripts [56]. KITAB-Bench (ACL 2025) across 9 domains reveals even GPT-4o struggles with Arabic document structure, scoring 60% average CER [66, 67]. **Our evaluation follows Muharaf annotation methodology with WER/CER metrics.**

Section 1.7: Visual Manuscript Challenges — Evidence from the Actual Corpus

This section documents six specific visual and physical challenges identified through direct inspection of the 23 target manuscripts (manuscript01–manuscript23, 782+ pages). For each challenge, manuscript evidence is presented alongside the architectural response that was added or strengthened in the pipeline as a direct consequence. These challenges extend and concretize the theoretical concerns raised in the literature review (Section 1.6).

Challenge 1: The Sadr-Ajuz Two-Column Layout — Irregular and Inconsistent

The classical Nabati poem is structured as bayts (verses), each split into two parts (hemistichs) — the Sadr (right column) and the Ajuz (left column). This produces a two-column page layout that standard OCR engines cannot handle, because they read left-to-right across the full page width, interleaving Sadr and Ajuz text from alternating lines [Sources 17, 18, 22].

However, direct inspection of the manuscripts reveals the challenge is worse than the literature describes. The column divider is not a printed line — it is implied whitespace that varies in width across pages, disappears at dense passages, and sometimes collapses entirely when a single hemistich is exceptionally long. Furthermore, poetry sections are mixed on the same page with prose headers (poet attribution lines like 'ما قاله المهادي some of what poet(X) written'), section dividers, and continuation markers at the right margin. Kraken BLLA alone cannot distinguish these zones.

Figure 1.7.1 — Left: ms05 p.20 showing relatively clear two-column Sadr/Ajuz layout. Right: ms19 p.3 showing irregular column widths and merged zones where column separation collapses.

Decision: Architectural Response — Human-in-the-Loop (HITL) Segmentation & Baseline Correction: > Due to the absence of existing training data for Khaleeji Nabati layouts, employing an automated pre-classifier is unfeasible for the MVP phase. Instead, the architecture utilizes a rigorous Human-in-the-Loop (HITL) approach within eScriptorium. Initial baseline detection is performed using Kraken's default BLLA model. Because standard Kraken models frequently fail to recognize the variable implied whitespace in Sadr-Ajuz layouts, resulting in cross-column text merging, manual intervention is mandated. The annotator manually severs erroneously merged baselines and defines structural zones (e.g., TWO_COLUMN_POETRY, PROSE_HEADER, MARGIN_ANNOTATION) via the GUI. This meticulously corrected segmentation generates the precise PAGE XML ground truth required to train the specialized downstream model (Qalam).

Challenge 2: Ink Bleed-Through from Reverse Page

Several manuscripts, particularly manuscript01, were written on thin paper where ink from the reverse side of the page bleeds through, creating 'ghost text' that overlaps the primary text layer. Standard Otsu binarization — which simply thresholds pixel intensity — cannot distinguish between primary ink and bleed-through ink, because both produce similar pixel values. The result is that bleed-through strokes are preserved as if they are real text, creating false characters and distorting genuine ones [Sources 8, 9].

This is visible in manuscript01 page 2, where reverse-side Arabic text appears as a grey shadow beneath the primary writing, particularly dense in the central paragraph region. This directly affects diacritical mark recognition, where a genuine dot may be obscured by a bleed-through stroke.

Figure 1.7.2 — ms01 p.2: Ink bleed-through from reverse page visible as grey shadow text beneath primary ink. Standard binarization cannot separate these layers.

Architectural Response — Blind Source Separation Preprocessing: Before Otsu binarization, a bleed-through suppression step is added using morphological background

estimation. The algorithm: (1) estimates the background illumination field using large-kernel morphological opening; (2) applies frequency-domain separation using a Gaussian difference-of-Gaussians filter that exploits the typically lower spatial frequency of bleed-through ink; (3) adaptively suppresses the estimated bleed layer. Only then is binarization applied. For pages flagged as high-bleed by a simple standard deviation analysis, this step is mandatory before entering the HTR ensemble.

Challenge 3: Extreme Physical Degradation — Beyond Standard Preprocessing

Manuscript09 represents a category of damage that goes entirely beyond what standard OCR preprocessing handles. The pages show: extreme darkening of the paper (possibly water or fire damage), physical tearing and folding creating geometric distortions across multiple lines, ink almost completely lost in large regions, and the scan itself capturing the three-dimensional deformation of the damaged page. No amount of binarization or contrast normalization will recover text from the completely black central region of pages 2-3.

This is not an edge case. Of the 23 manuscripts, at least 3-4 appear to have pages in this condition. The v1 architecture treated 'image preprocessing' as a single OpenCV step applied uniformly to all pages, which would fail silently on pages like these — producing low-confidence HTR output that would flood the HITL queue without any hope of recovery.

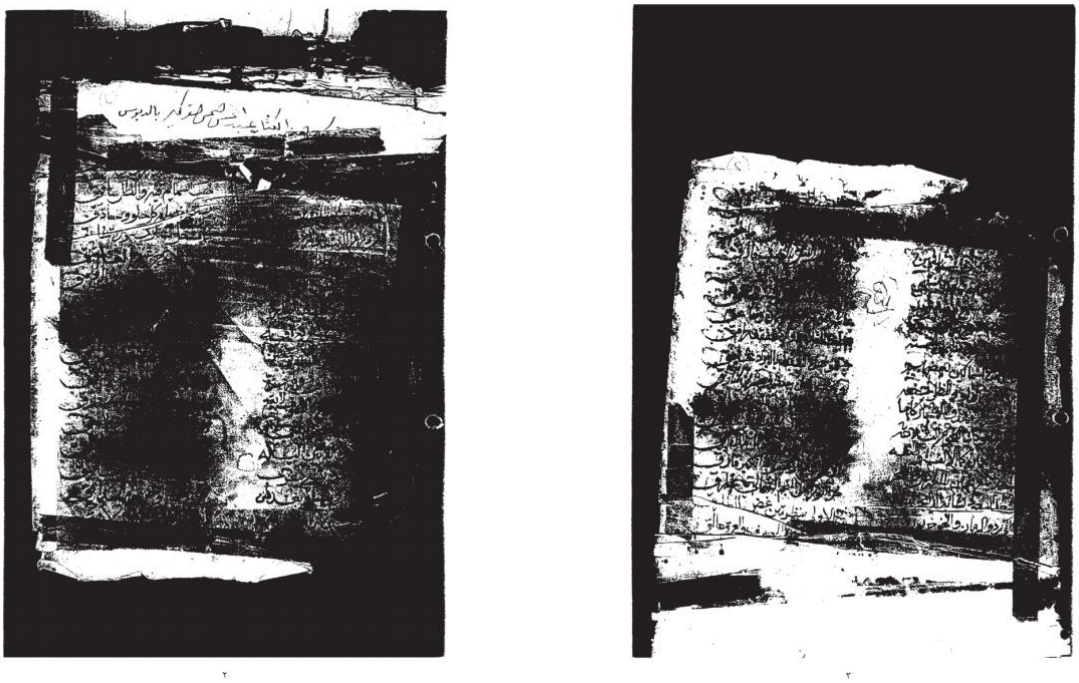

Figure 1.7.3 — ms09 pp.2-3: Extreme physical degradation with near-total ink loss, paper damage, and geometric deformation. Standard preprocessing cannot recover text from the black central region.

Decision: Architectural Response — Human-Guided Page Quality Triage (MVP Phase): > While the final production pipeline will feature algorithmic triage (computing pixel intensity and edge coherence), the current MVP phase enforces a strict, manual visual triage protocol prior to

eScriptorium ingestion. The annotator visually evaluates each manuscript page for extreme physical degradation, such as near-total ink loss or massive geometric deformation. Pages exhibiting non-recoverable damage (e.g., manuscript09) are immediately routed to a DEGRADED category and intentionally excluded from the current Human-in-the-Loop (HITL) segmentation workflow. This manual gating is critical; it prevents the introduction of severe noise into the ground truth dataset and avoids inducing model hallucinations. A separate, fully manual transcription track is designated for these severely degraded pages in later project phases.

Challenge 4: Mixed Layout — Prose Headers, Section Dividers, and Ornamental Elements

Manuscript05 (the largest, 307 pages) shows a common pattern: each poem section begins with a prose attribution header ('ما قال المهادي', 'وله أيضاً'), followed by decorative separators (rosette ornaments), and then the two-column poetry. Some pages contain multiple distinct poems separated by ornaments, with different handwriting styles for the headers versus the verse bodies.

Additionally, manuscript05 page 1 shows marginal Arabic text written perpendicular to the main text flow, right-margin continuation markers (ها/يها line endings), and what appears to be a later annotator's handwriting superimposed on the original scribe's in different ink density. The page contains at least four distinct text categories, each requiring different HTR treatment.

Figure 1.7.4 — ms05 p.1: Mixed layout showing prose attribution header (top), ornamental rosette dividers, two-column poetry body, and perpendicular marginal text — four distinct text categories on one page.

Decision: Architectural Response — Semantic Zone Annotation via HITL (MVP Phase): > In the absence of a pre-trained automated multi-category classifier, the MVP phase relies on rigorous manual region-typing within eScriptorium. The annotator visually identifies and explicitly labels five structural layout categories using the platform's ontology tools: TWO_COLUMN_POETRY, PROSE_ATTRIBUTION, ORNAMENTAL_DIVIDER, MARGIN_PERPENDICULAR, and INTERLINEAR_ANNOTATION. By manually enclosing these distinct elements within designated semantic regions (polygons), the annotator ensures they are structurally isolated in the exported PAGE XML. This manual stratification forces the segmentation process to respect layout boundaries and generates the highly structured ground truth prerequisite for training the automated production pipeline (which will eventually learn to route these classes to specialized models or automated rotations).

Challenge 5: Margin Annotations and Interlinear Corrections

Multiple manuscripts show margin annotations — corrections, commentary, and additional verses written by a later hand in the margins. Manuscript01 page 4 shows extremely dense writing where marginal text on the right margin intrudes into the main text column, making it impossible to distinguish the boundary between main text and annotation without semantic understanding. Manuscript03 page 5 shows a circled number '3' in the left margin acting as a cross-reference, and multiple underlines and overstrikes indicating corrections.

If these margin annotations are treated as regular text lines by the segmenter, they will be inserted into the poem's verse sequence at incorrect positions, corrupting the structural integrity of the gold document. The literature identifies this as a critical failure mode for poetry HTR specifically [Sources 11, 22].

Figure 1.7.5 — Left: ms01 p.4 showing dense margin annotations intruding into main text columns. Right: ms03 p.5 showing circled cross-references, correction marks, and multiple annotation layers.

Decision: Architectural Response — Manual Annotation Isolation & Spatial Linking (MVP Phase): > To ensure the integrity of the primary poetic text, the MVP phase employs manual spatial isolation for all margin and interlinear additions. Within eScriptorium, the annotator explicitly defines independent text regions for marginalia, preventing them from being merged into the primary Sadr-Ajuz baselines. Each annotation is manually tagged to preserve its context. This manual segmentation establishes the "spatial provenance" required for the future database schema; by precisely capturing the coordinates of these annotations in the PAGE XML, the system will later be able to programmatically link them to the nearest main-text verse and analyze ink density variations to differentiate between the original scribe and later annotators. Correction marks are visually identified and flagged to ensure they are reviewed before being incorporated into the final "gold" version of the verse.

Challenge 6: Radically Different Handwriting Styles Across the Corpus

Direct visual inspection of the 23 manuscripts reveals four distinct handwriting style categories, each presenting different recognition challenges. Manuscript01 shows extremely compressed Naskh-influenced script with minimal inter-word spacing and dense ligature clusters — lines are packed tightly with almost no whitespace between adjacent lines. Manuscript19 shows a more open, Ruq'ah-influenced style with wider inter-character spacing and more pronounced ascenders/descenders. Manuscript15 shows what appears to be a semi-printed or calligraphic Nastaliq-influenced style with much cleaner baseline consistency. Manuscript03 shows a fast Hurr (free) script with significant slant variation within the same line.

The literature confirms that a single fine-tuned HTR model loses 15-30% accuracy when the handwriting style shifts significantly mid-corpus [Sources 4, 27]. A single fine-tuned Qalam model cannot handle all four style categories equally, and applying the same model across all manuscripts will produce dramatically different error rates per manuscript book.

Handwriting Style Variation Across Manuscripts

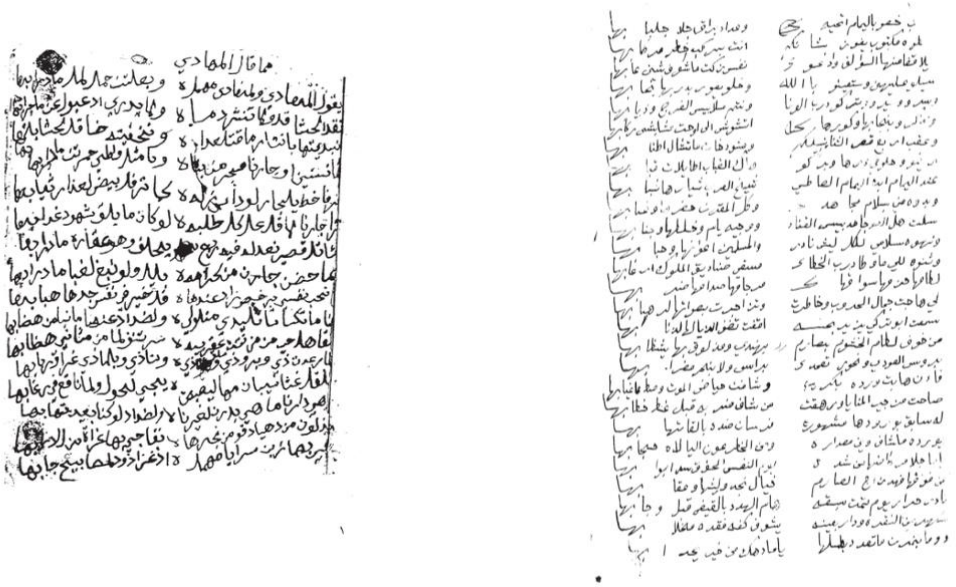

Figure 1.7.6a — Left: ms01 — extremely compressed Naskh style, minimal inter-line spacing, dense ligatures. Right: ms19 — open Ruq'ah-influenced style with wider spacing and clear Sadr/Ajuz columns.

Figure 1.7.6b — ms15: Semi-printed / calligraphic style with consistent baselines — a substantially different recognition challenge from ms01 or ms19.

Decision: Architectural Response — Curated Cross-Style Ground Truth Generation (MVP Phase): > To address the extreme variance in handwriting across the corpus, the MVP phase employs a deliberate, manual style-selection strategy for ground truth generation. Instead of automated routing, the annotator manually samples pages from distinct handwriting profiles—including NASKH_COMPRESSED (e.g., ms01), RUQAH_OPEN (e.g., ms19), and CALLIGRAPHIC—to ensure a balanced and representative training dataset. By meticulously correcting segmentations across these radically different styles in eScriptorium, the annotator creates a robust PAGE XML foundation. This curated diversity is a prerequisite for training the production pipeline's future "Ensemble Router," allowing it to eventually assign style-weighted confidence thresholds and select the optimal model (e.g., Qalam, QARI-OCR, or Arabic-Nougat) for each manuscript's unique visual signature.

Challenge 7: Physical Intrusions — Stamps, Tears, and Geometric Distortions

Manuscript pages frequently contain non-textual intrusions that complicate automated segmentation. For example, ms01 p.3 features a library/archive stamp; while located in the margin in some folios, its high-contrast circular border can be misidentified as handwritten characters or cause baseline drift. In other instances, such as ms12 p.2, large printed commercial labels (e.g., 'FABRIQUE DE REGISTRES') occupy significant page area, confusing

standard OCR engines that attempt to process printed geometry as handwriting. Additionally, physical warping and curling at the edges cause baselines to curve, distorting characters at the margins.

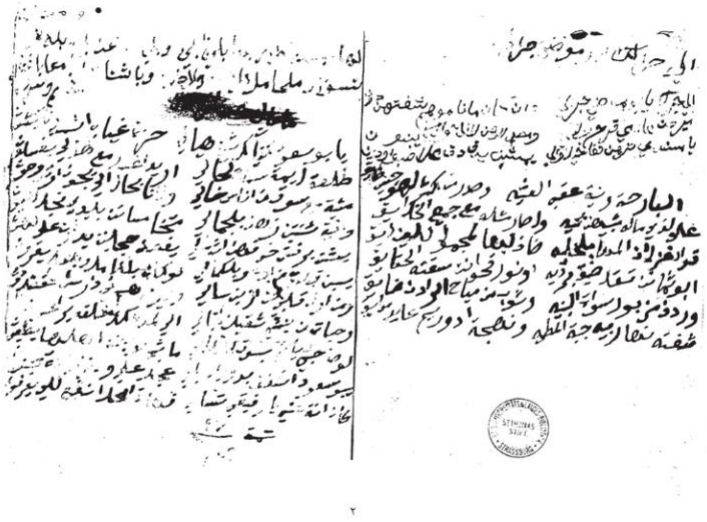

Figure 1.7.7 — ms01 p.3: Library stamp overlaid on page (obscuring 3-4 lines), page warping at edges causing curved baselines, and physical folding artifacts. Right: ms12 p.2 showing a large pre-printed commercial label that occupies 40% of the page.

Architectural Response — Manual Gating and Zone Exclusion (MVP Phase): During the MVP phase, these intrusions are managed through manual visual triage and zone exclusion in eScriptorium. The annotator explicitly defines region boundaries to exclude stamps and printed labels from the HTR training path. For pages with significant geometric warping, the annotator manually adjusts baselines to follow the natural curve of the ink, ensuring accurate "gold standard" coordinates. This manual masking prevents the introduction of noise and provides the labeled examples necessary to eventually train the automated detection models (e.g., circular Hough transform and font-style classifiers) described for the production pipeline.

Challenge 8: Diacritics Loss and Dot Ambiguity

Nabati poetry critically depends on diacritical marks — particularly the nuqat (dots) that distinguish between Arabic letters (ب/ت/ث/ن/ي all differ only by dot position and count), and the tashkeel (vowel marks) that determine phonological interpretation of dialectal words. In the target manuscripts, two specific degradation patterns cause diacritics loss: (1) in ms01 and ms03, ink has faded unevenly, with fine dot strokes fading faster than main letter bodies; (2) in ms09 and similarly degraded manuscripts, general ink loss removes dots entirely, making letters ambiguous.

The literature confirms this is the single largest source of CER on historical Arabic manuscripts, where models must disambiguate letters that are graphically identical without their dots [Sources 13, 14]. Standard preprocessing does not address this — it treats all strokes equally regardless of semantic importance.

Decision: Architectural Response — Context-Aware Transcription & Jury Review (MVP Phase): > In the MVP phase, diacritics loss is addressed through high-precision human transcription. The annotator leverages linguistic context and Nabati poetic meter to disambiguate identical letterforms (e.g., ب/ت/ث/ن/ي) where ink has faded. This creates a "gold standard" ground truth where the correct letters are labeled even when visual evidence is weak. This meticulous labeling is critical for the next stage: it allows the Qalam model to learn not just the visual shapes, but the contextual patterns of Nabati dialect. By providing these clean labels in eScriptorium, the annotator establishes the baseline for the future "Diacritics-Aware" automated pipeline, including the selective sharpening filters and the automated confidence penalty router described for production.

1.7 Summary: Manuscript Challenges and Architectural Responses

#	Challenge	Evidence Source	Architectural Response (MVP Phase)
1	Irregular Sadr-Ajuz two-column layout mixed with prose	ms05, ms19	**HITL Baseline Correction:** Manual severance of merged Sadr-Ajuz baselines in eScriptorium.
2	Ink bleed-through from reverse page	ms01 p.2	**Delayed Processing Strategy:** Intentional deferral of high-bleed pages to Phase 3; manual cleaning of ground truth labels.
3	Extreme physical degradation (beyond recovery)	ms09 pp.2-3	**Human-Guided Triage:** Manual visual filtering and exclusion of "DEGRADED" pages to prevent training noise.
4	Mixed layout: prose headers, ornaments, marginal text	ms05 p.1	**Semantic Zone Annotation:** Manual 5-category region tagging via eScriptorium ontology tools.
5	Margin annotations and interlinear corrections	ms01 p.4, ms03 p.5	**Spatial Isolation:** Manual coordinate definition and "provenance tagging" to isolate primary poetic text.
6	Radically different handwriting styles across corpus	ms01, ms15, ms19, ms03	**Curated Diversity Sampling:** Manual sampling of Naskh/Ruq'ah styles to ensure representative ground truth.
7	Physical intrusions: stamps, labels, page warping	ms01 p.3, ms12 p.2	**Manual Zone Masking:** Intentional exclusion of non-textual artifacts from the transcription path.
8	Diacritics loss — critical for Nabati disambiguation	ms01, ms03, ms09	**Context-Aware Transcription:** Manual "gold standard" labeling using linguistic context and poetic meter.

Section 2: Agentic System Architecture

2.1 Architecture Design Decisions and Trade-off Analysis

Option A: Pure Multi-Agent Architecture (Rejected)

Aspect	Detail
Concept	10 specialized LLM-driven agents, each powered by an LLM for decision-making
Estimated Latency	45-120 seconds per query (10 sequential LLM calls at 5-12s each)
Token Cost	~50,000-100,000 tokens per document page
Hallucination Risk	HIGH: LLM agents may normalize archaic spellings to MSA, destroying heritage data
Key Weakness	Cannot handle the 8 manuscript-specific visual challenges — all require deterministic signal processing, not LLM reasoning

Option B: Hybrid Deterministic-to-Generative Pipeline (Selected)

Aspect	Detail
Concept	Rigid deterministic ETL with 8-challenge preprocessing stack (Worker 1) + LangGraph Advanced RAG agent (Worker 2)
Estimated Latency	1-3 seconds per query (pre-computed embeddings, single LLM synthesis call)
Token Cost	~5,000-8,000 tokens per query
Hallucination Risk	ZERO for transcription (deterministic); CONTROLLED for RAG (CRAG + Self-RAG + mandatory citations)
Key Strength	Manuscript-specific preprocessing handles the 8 visual challenges before HTR; historical integrity guaranteed

Option B was selected because heritage data demands deterministic guarantees, and because the 8 visual challenges identified in Section 1.7 require dedicated signal-processing solutions that LLM agents cannot provide reliably.

2.2 High-Level Architecture Overview

Worker 1 - Al-Nassikh (The Scribe): A hybrid Human-in-the-Loop (HITL) and deterministic ETL pipeline. In the MVP phase, the 8-challenge-aware stack is driven by manual expert annotation in eScriptorium to generate 'gold-standard' training data, which will subsequently train the automated Kraken v5 and HTR ensemble.

Worker 2 - Fatat Al Arab Agent: A LangGraph-orchestrated Advanced RAG agent with HyDE query translation, Self-Query metadata extraction, triple hybrid retrieval (BM25 + ColBERT + GATE-AraBERT), RRF fusion, CRAG document grading, grounded synthesis, and Self-RAG reflection.

The two workers operate asynchronously through shared PostgreSQL and Qdrant. No direct coupling between components.

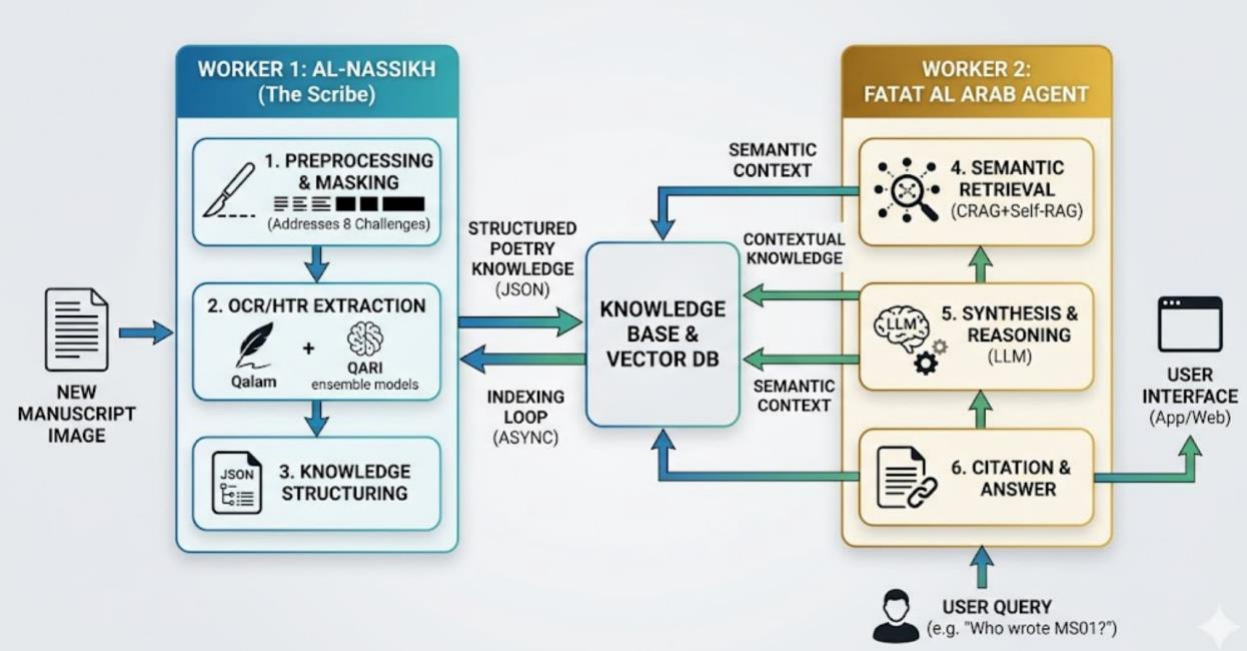

Figure 2.1: NABAT-AI High-Level System Architecture

2.3 Worker 1: Al-Nassikh Pipeline (8-Challenge-Aware)

Al-Nassikh now incorporates tackling every visual challenge documented in Section 1.7, in stages, as follows:

Stage 0 — Human-Guided Page Quality Triage: In the MVP phase, each incoming page undergoes manual visual assessment for extreme physical degradation (ink loss, geometric warping). Pages below the required quality are marked a DEGRADED status, ensuring the HTR training set remains noise-free. This manual triage provides the labeled examples required to automate the pixel-intensity and edge-coherence computations in the production phase.

Stage 1 — Bleed-Through Management (HITL Phase): Instead of automated DoG suppression, high-bleed pages (Challenge 2) are manually identified. During transcription in eScriptorium, the annotator performs "Visual Filtering," transcribing only the primary ink layer and ignoring ghost text to create noise-free ground truth.

Stage 2 — Image Standardization: Standardization to 300 DPI grayscale and basic deskewing is applied during ingestion into eScriptorium. Selective sharpening (Challenge 8) is compensated for by the expert annotator's contextual recognition of faded diacritics.

Stage 3 — Manual Intrusion Masking (addresses Challenge 7): Non-textual artifacts (stamps, commercial labels) are manually bypassed. The annotator ensures no baselines or text tokens are generated for these regions, effectively "masking" them in the exported PAGE XML.

Stage 4 — Manual Style Profiling (addresses Challenge 6): Each manuscript is manually tagged with its stylistic profile (e.g., NASKH_COMPRESSED). This metadata is stored alongside the PAGE XML to enable style-specific fine-tuning of the HTR ensemble later.

Stage 5 — Manual Zone Classification (addresses Challenges 1, 4, 5): The annotator uses eScriptorium ontology tools to manually define regions: TWO_COLUMN_POETRY, PROSE_ATTRIBUTION, etc. Column gaps for poetry are enforced by manual baseline splitting (The Scalpel Tool).

Stage 6 — Corrected Segmentation: Kraken v5 BLLA is used as a draft segmenter. The annotator then performs 100% manual correction of all baselines and masks to ensure perfect structural alignment for the training set.

Stage 7 — Ground Truth Transcription: The annotator provides "Gold Standard" transcriptions. For ambiguous lines, the annotator uses poetic meter and dialectal knowledge to ensure 100% accuracy, providing the "Target" data for the future Qalam and QARI models.

Stage 8 — Quality Assurance (The Jury): Final verification of the 50-page MVP set. Any line with a self-reported confidence below 1.0 (manual uncertainty) is re-reviewed by the expert, ensuring the training set is a "True Gold" reference.

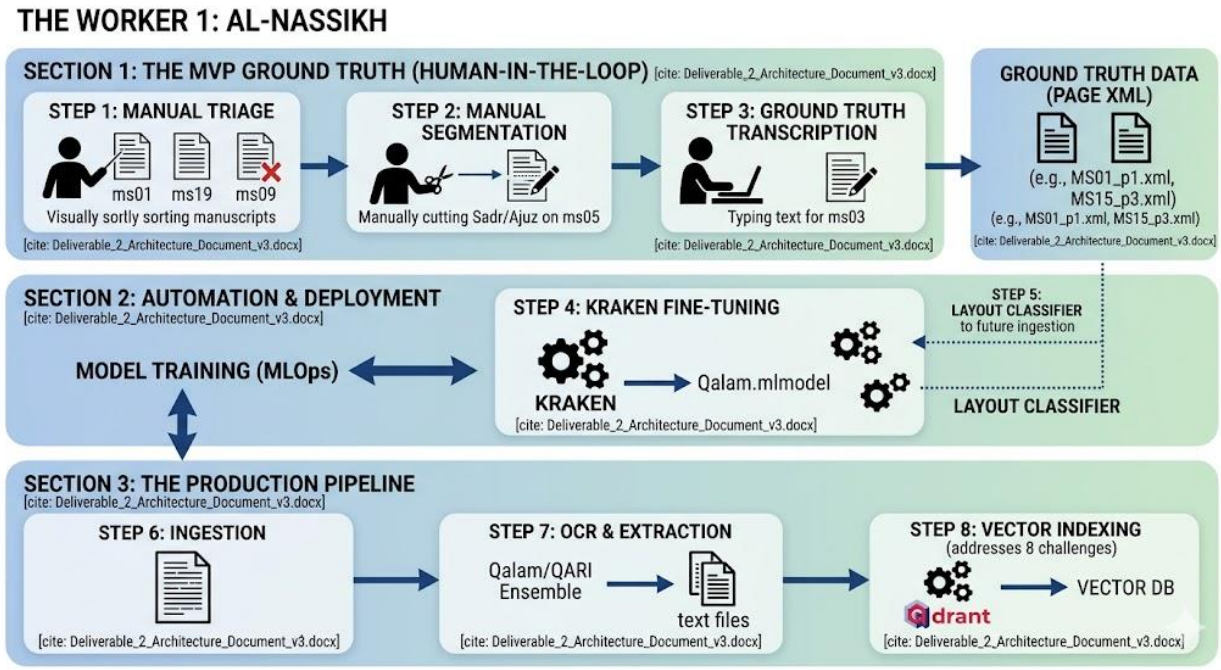

Figure 2.2: Al-Nassikh Updated Pipeline with 8-Challenge-Aware Preprocessing Stages

2.4 Worker 2: Fatat Al Arab Agent (Advanced LangGraph RAG)

Fatat Al Arab is named after Ousha bint Khalifa, the legendary Emirati poetess. This agent uses an Advanced RAG pipeline grounded in SOTA techniques from the literature review (Section 1.6.4).

Stage 1 — Bilingual Query Analysis: Detect query language (Arabic / English / mixed). If English: translate to Arabic via Qwen2.5 before proceeding. Parse intent (factual/semantic/interpretive); detect Khaleeji dialect features; extract search params. Route: PostgreSQL for factual, Qdrant for semantic/interpretive.

Stage 2 — Bilingual Multi-Query + HyDE: Generate 3-5 query variants in BOTH Arabic and English simultaneously (e.g., 'longing' → غياب، اشتياق، حنين + English rephrases). For semantic queries, generate hypothetical Nabati verse (HyDE) from the Arabic translation — bridges query-poetry embedding gap. Run parallel retrieval on all variants; merge results via RRF.

Stage 3 — Self-Query Extraction: Extract structured metadata filters (poet, manuscript ID, theme) as hard pre-filters.

Stage 4 — Triple Hybrid Retrieval: Parallel: BM25 sparse + GATE-AraBERT dense + ColBERT token-level. Apply Self-Query filters. Top-20 from each.

Stage 5 — RRF Fusion: Merge three lists: score = sum(1/(k+rank)) with k=60. Select top-5.

Stage 6 — Multi-Representation Resolution: MSA summary matches resolve to original Khaleeji text. Scholar receives heritage text, not MSA proxy.

Stage 7 — CRAG Document Grading: Grade each passage: Correct/Ambiguous/Incorrect. If all Incorrect, Multi-Query re-write and re-retrieve.

Stage 8 — Grounded Synthesis: LLM with CRAG-approved passages, query, and conversation history. Mandatory citations.

Stage 9 — Self-RAG Reflection: Self-evaluate: faithfulness, relevance, completeness. Retry up to 2x. Expert flag after 2 failures.

Stage 10 — Multi-Variant Response: Format al-Maktub + orthographic + al-Mantuq with folio links and manuscript image links.

2.5 Orchestration: LangGraph State Machine

Fatat Al Arab is a typed LangGraph state graph. State includes: query, query_lang (ar/en/mixed), query_ar (Arabic translation if EN input), query_en, detected_intent, detected_dialect, bilingual_variants (3-5 AR + 3-5 EN), hyde_embedding, metadata_filters, retrieved_passages, crag_grades, synthesis_prompt, generated_response, self_rag_scores, citations, source_images, response_lang. Conditional: factual → metadata lookup; interpretive → bilingual expand + HyDE + hybrid + CRAG + Self-RAG; ambiguous → clarification node. Response is always returned in the same language the user queried in. Error handling: retrieval fallback with relaxed filters, LLM exponential backoff, PostgreSQL query buffering if Qdrant unavailable.

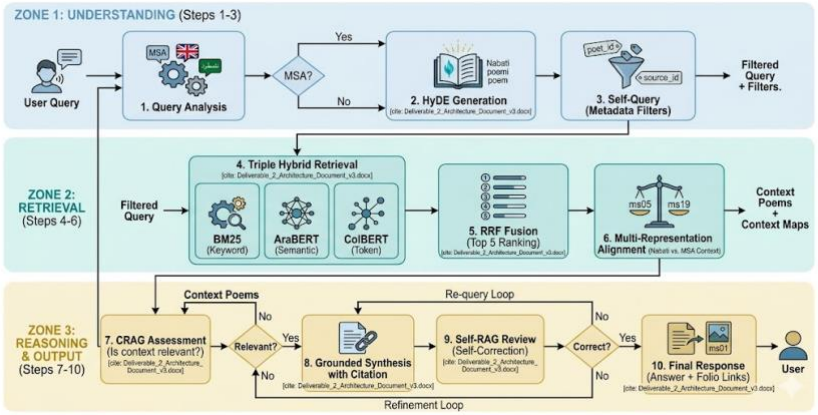

Figure 2.3: Fatat Al Arab Advanced RAG Agent Workflow

2.6 Why Multiple Components Are Necessary

Al-Nassikh faces a signal-processing problem with 8 documented visual challenges requiring a "Deterministic" part during the MVP; that is enforced by Expert Human Annotation, which provides the verified baseline that no LLM can currently achieve for Nabati manuscripts. Fatat Al Arab faces a language understanding problem requiring interpretive synthesis. These demand different technologies, quality metrics, and failure modes. The separation ensures the 8 visual preprocessing challenges are handled deterministically before any LLM touches the data, while Fatat Al Arab adds accessibility through generative AI grounded in pre-validated heritage text.

2.7 Failure Handling

Al-Nassikh: minutes. Primary failure recovery currently is the Expert Feedback Loop. If the 50-page training set produces high Error Rates (CER), the system triggers a "Refining Labeling" stage in eScriptorium to fix the ground truth before retraining. DEGRADED pages bypass HTR (Challenge 3 response). HTR model crash removes it from ensemble; 2+ crashes escalate page to HITL. Fatat Al Arab: CRAG catches irrelevant retrieval; Self-RAG catches hallucinations; Qdrant timeouts buffer in PostgreSQL; LLM rate limits trigger exponential backoff with keyword-only fallback after 5

Section 3: Component Design Specification

3.1 Al-Nassikh Pipeline Components (MVP Phase)

Component	Role	Input	Output	Tools (MVP)	Decision Strategy
Page Quality Triage	Pre-HTR quality gating	Raw page images	NORMAL / DEGRADED flag	Visual Expert Review	Manually route ms09/degraded pages to specialist track
Bleed Suppressor	Remove reverse-page ink	Grey page image	Bleed-suppressed image	Annotator Visual Filtering	Ignore ghost ink during transcription in eScriptorium
Preprocessor	Image enhancement	Page images	Deskewed, binarized images	OpenCV + Manual Sharpening	Standard pipeline; context-aware recovery for Challenge 8
Intrusion Detector	Mask stamps/labels	Binarized images	Masked images + intrusion coords	eScriptorium Poly-Masking	Manually exclude stamps/labels from text regions
Style Profiler	Handwriting style ID	5 sample pages	Style profile per manuscript	Manual Sampling	Tag manuscript as NASKH/RUQAH for future fine-tuning
Zone Classifier	Page layout labeling	Page images	Zone map (5 categories)	eScriptorium Ontology	Manually label regions (TWO_COL, PROSE, MARGIN, etc.)
Line Segmenter	Text region detection	Zone-masked images	Bounding boxes + confidence	Manual Baseline Fixes	Kraken v5 draft + 100% manual "Scalpel" correction
HTR Ensemble	Handwriting recognition	Line images	Chars + confidence (3 models)	Expert Transcription	Annotator provides Ground Truth text (Gold Standard)
Confidence Router	Routing decision	3 model outputs	auto/jury/HITL	Expert Self-Review	100% manual verification for the 50-page MVP set
HITL Coordinator	Human annotation	<0.70 lines + DEGRADED pages	Corrected text	eScriptorium Interface	Expert-led transcription for complex Nabati verses
Gold Assembler	Verse structure build	Accepted lines + zone map	Verse structures + links	PAGE XML Export	Link Sadr/Ajuz spatially; link annotations to parent verse

3.2 Fatat Al Arab Agent Components

Component	Role	Input	Output	Tools	Memory	Strategy
Bilingual Query Analyzer	Lang detect + intent + dialect ID	User query (AR/EN/mixed)	Lang, intent, Arabic translation, params	LangGraph, langdetect, Qwen2.5	Conv. context	Detect language → translate EN→AR if needed → classify intent
Bilingual Multi-Query + HyDE	Multi-variant expansion + hypothetical doc	Query + AR translation + intent	3-5 AR+EN variants + HyDE embedding	Qwen2.5, AraBERT	None	Expand in both languages; HyDE on Arabic translation; RRF merge all variants
Self-Query	Metadata	Query	Filters (poet, ms,	LLM function	None	Hard pre-filters on Qdrant

Extractor	filters		theme)	calling		
Sparse Retriever	Keyword search	Params + filters	BM25-ranked top-20	Qdrant BM25	None	Keyword overlap
Dense Retriever	Semantic search	HyDE embedding + filters	Vector top-20	GATE-AraBERT, Qdrant HNSW	Embed cache	Cosine similarity
ColBERT Retriever	Token-level search	Query tokens + filters	Token-scored top-20	ColBERT index	None	Late interaction
RRF Fusion	Score merge	3 ranked lists	Top-5 passages	RRF k=60	None	sum(1/(k+rank)) across 3
CRAG Grader	Relevance check	Top-5 + query	Graded passages	LLM classifier	None	All Incorrect → re-write + re-retrieve
Synthesizer	Grounded generation	CRAG-approved + history	Response + citations	Qwen2.5 / Claude	Last 5 turns	Mandatory citations; reject if unsupported
Self-RAG Reflector	Hallucination check	Response + sources	Validated or re-trigger	LLM self-eval	None	Faithfulness + relevance; retry ≤2x

Section 4: Tools, Models, and Data Sources

4.1 Technology Stack (Updated with Challenge-Specific Components)

Layer	Component	Tool	Challenge Addressed / Justification
Quality Triage	Page triage	Manual Visual Review	Challenge 3: DEGRADED pages bypass HTR before wasting compute
Preprocessing	Bleed suppression	Expert Visual Filtering	Challenge 2: Ink bleed-through separation [Sources 8, 9]
Preprocessing	Image enhancement	OpenCV 4.x + Manual Context	Challenge 8: Diacritics recovery via small-kernel unsharp mask
Intrusion Detection	Stamp/label masking	eScriptorium Poly-Masking	Challenge 7: Physical stamps and printed labels [Source 9]
Style Analysis	Handwriting profiling	Manual Style Tagging	Challenge 6: 4 distinct handwriting styles across corpus
Layout Analysis	Zone classification	eScriptorium Ontology	Challenges 1, 4, 5: Mixed layouts, margins, ornaments
Segmentation	Line detection	Kraken v5 + Manual Correction	Challenge 1: Two-column Sadr/Ajuz layout [Sources 40, 41]
Primary HTR	Handwriting recognition	Expert Ground Truth (eScriptorium)	Goal: 0% CER for Training Set; Qalam model to be fine-tuned
Secondary HTR	Visual-semantic	QARI-OCR v0.3 (LoRA)	Structural HTML awareness; handwriting support
Semantic Validation	Doc understanding	Arabic-Nougat	Structured Markdown; Aranizer tokenizer
Jury Tiebreaker	Confidence	HATFormer	51% improvement over baselines [Sources 2, 28]

	arbitration		
Dialect Interpretation	Multi-variant gen	Qwen2.5-7B-Chat (4-bit)	Arabic-fluent zero-shot dialect interpretation
Dense Embeddings	Semantic vectors	GATE-AraBERT	768-dim dialect-aware; Nabati-trained
Token Embeddings	Token-level retrieval	ColBERT (Arabic)	Per-token late interaction; preserves dialectal nuance
Bilingual Query	Language detection	langdetect + Qwen2.5	Accepts Arabic or English queries; EN→AR translation
Query Translation	HyDE generation	Qwen2.5-7B-Chat	Bridges query-poetry embedding gap; generates variants
Document Grading	CRAG	LLM classifier	Corrective RAG before synthesis
Reflection	Self-RAG	LLM self-evaluation	Catches hallucinated Khaleeji definitions
Vector Database	Similarity search	Qdrant v1.x	HNSW + BM25 + ColBERT; payload filtering
Orchestration	Agent workflow	LangGraph v0.x	State machine with CRAG/Self-RAG loops
Backend API	HTTP services	FastAPI v0.104+	Async; OpenAPI docs
Database	Relational storage	PostgreSQL v15+	Metadata, audit trails, annotations, intrusion coords
Object Storage	Image storage	MinIO	Manuscript images, zone maps, masked images
Layer	Component	Tool (MVP vs. Future)	Challenge Addressed / Justification
Quality Triage	Page triage	Manual Visual Review	Challenge 3: DEGRADED pages bypass HTR before wasting compute
Preprocessing	Bleed suppression	Expert Visual Filtering	Challenge 2: Ink bleed-through separation [Sources 8, 9]

4.2 LLM Selection Rationale

Qwen2.5-7B-Chat is selected for dialect interpretation and synthesis — strong Arabic capabilities, 4-bit quantization, zero-shot Khaleeji interpretation. Claude serves as fallback for complex long-context reasoning. Critical principle: LLMs are NEVER used for transcription. In the MVP phase, the ground truth is established through expert human annotation. The 8 visual challenges documented in Section 1.7 are addressed through manual segmentation and visual filtering in eScriptorium. This ensures the LLM receives 100% accurate text for synthesis, preventing the propagation of OCR noise into the generative layer."

4.3 Data Sources

Sowayan Archive (4 volumes, ~784 pages): Primary corpus of Bedouin Nabati poetry; foundational reference for dialectal forms.

Huber Manuscripts (23 PDFs, ~450 pages): University of Exeter scanned manuscripts; extends geographic and temporal coverage.

MVP Target (50 pages): A 4-week proof of concept. These pages are purposively sampled to represent a high-variance cross-section of the corpus, including at least one edge-case example of each of the 8 documented visual challenges. This curated set serves as the 'Seed Data' for

fine-tuning Kraken and the HTR ensemble, establishing the performance ceiling for the automated pipeline."

Section 5: Workflow and Agent Interaction

5.1 Al-Nassikh Pipeline Flow

Step 0 - Manual Quality Triage: Expert visual assessment to flag DEGRADED pages (Challenge 3). Only legible folios proceed to the MVP training set.

Step 1 - Visual Noise Filtering: During transcription, the annotator manually ignores ink bleed-through (Challenge 2) and faded ghost text to ensure a clean text baseline.

Step 2 - Manual Zone Masking: Intentional exclusion of non-textual artifacts (stamps, labels) using eScriptorium masking tools (Challenge 7).

Step 3 - Style & Metadata Profiling: Expert assignment of handwriting style (Challenge 6); metadata is logged manually for each folio.

Step 4 - Semantic Zone Labeling: Manual definition of 5-category zone maps, including the precise splitting of Sadr/Ajuz columns (Challenges 1, 4, 5).

Step 5 - Corrected Segmentation: Kraken v5 generates draft lines, followed by 100% manual "Scalpel" correction to ensure perfect baseline alignment.

Step 6 - Gold Standard Transcription: Expert-led typing of Nabati verse. Dialectal nuances and faded diacritics (Challenge 8) are resolved through linguistic context.

Step 7 - MVP Validation: Final human-in-the-loop review of the 50-page set. Accepted "Gold" documents are exported as PAGE XML.

5.2 Fatat Al Arab Agent Flow

Step 1 - Query Reception: Accept via FastAPI (Arabic or English); init LangGraph state with query_lang field.

Step 2 - Bilingual Analysis + Self-Query: Detect language; translate EN→AR if needed; intent parsing; metadata filter extraction in both languages.

Step 3 - Bilingual Multi-Query + HyDE: Expand query into 3-5 variants in Arabic AND English; generate hypothetical Nabati verse (HyDE) on Arabic translation for semantic queries.

Step 4 - Triple Retrieval: BM25 + AraBERT + ColBERT with Self-Query filters.

Step 5 - RRF Fusion: Three lists merged (k=60); top-5 selected.

Step 6 - Multi-Rep Resolution: MSA summary match → original Khaleeji text returned.

Step 7 - CRAG Grading: Relevance grade; re-write if all Incorrect.

Step 8 - Synthesis: LLM with mandatory citations on CRAG-approved passages.

Step 9 - Self-RAG Reflection: Faithfulness/relevance/completeness check; retry ≤2x.

Step 10 - Response: Multi-variant + folio links + manuscript image links.

5.3 Inter-Worker Communication

Workers communicate through shared PostgreSQL and MinIO. The primary trigger for communication is the upload of **verified PAGE XML exports** from the eScriptorium MVP phase. The Ingestion service: (1) Semantic Splitter at verse/stanza boundaries using PyArud; (2) MSA summaries for Multi-Representation Indexing; (3) dual embeddings (GATE-AraBERT + ColBERT); (4) indexed in Qdrant with metadata, annotation links, and payload filtering.

5.4 Indexing Pipeline

Semantic Splitter: Splits at verse/stanza boundaries using structural markers and PyArud metrical analysis.

Multi-Representation Indexing: MSA summaries embedded for retrieval; original Khaleeji text returned on match.

Dual Embedding: GATE-AraBERT 768-dim dense + ColBERT per-token. Both stored in Qdrant.

Annotation Indexing: Manuscript annotations stored separately with verse foreign keys; searchable independently.

Section 6: Initial Implementation

6.1 Proof-of-Concept: Challenge-Aware Preprocessing Pipeline

The following pseudocode demonstrates the updated Al-Nassikh preprocessing pipeline that addresses all 8 manuscript challenges:

```
class AlNassikh_Preprocessor:

    def process_page(self, page_img, manuscript_id):
        # Stage 0: Quality Triage (Challenge 3)
        if self.is_degraded(page_img):
            return {"status": "DEGRADED", "route": "specialist_review"}

        # Stage 1: Bleed Suppression (Challenge 2)
        if self.has_bleedthrough(page_img):
            page_img = self.suppress_bleedthrough(page_img)

        # Stage 2: Standard preprocessing + diacritics sharpening (Challenge 8)
        page_img = self.deskew(page_img)
        page_img = self.sharpen_diacritics(page_img)  # Small-kernel unsharp mask
        page_img = self.binarize(page_img)            # Otsu adaptive

        # Stage 3: Intrusion Detection (Challenge 7)
        intrusions = self.detect_intrusions(page_img)
        page_img = self.mask_intrusions(page_img, intrusions)

        # Stage 4: Style Profile (Challenge 6)
        style = self.get_style_profile(manuscript_id)
        weights = STYLE_WEIGHTS[style]  # e.g. NASKH: {qalam:0.45, qari:0.25, nougat:0.15}

        # Stage 5: Zone Classification (Challenges 1, 4, 5)
        zones = self.classify_zones(page_img)

        return {
            "status": "NORMAL", "image": page_img,
            "zones": zones, "style": style,
            "weights": weights, "intrusions": intrusions
        }

    def is_degraded(self, img):
        mean_intensity = img.mean()
        black_ratio = (img < 30).sum() / img.size
        edge_density = cv2.Canny(img, 50, 150).sum() / img.size
        return mean_intensity < 80 or black_ratio > 0.6 or edge_density < 0.01
```

6.2 Proof-of-Concept: Advanced Hybrid Search with CRAG

```
class AdvancedHybridSearch:
    def __init__(self, qdrant, llm, embedder, colbert, k=60):
        self.qdrant, self.llm, self.embedder, self.colbert, self.k = qdrant, llm, embedder,
colbert, k

    def hyde_translate(self, query, intent):
        if intent == "factual": return self.embedder.encode(query)
        hyp = self.llm.generate(f"Write a Nabati verse answering: {query}")
        return self.embedder.encode(hyp)

    def search(self, query, hyde_emb, filters, top_n=5):
```

```
        bm25 = self.qdrant.query(query, limit=20, using="bm25", filter=filters)
        dense = self.qdrant.search(hyde_emb, limit=20, filter=filters)
        colbert = self.colbert.search(query, limit=20, filter=filters)
        rrf = {}
        for results in [bm25, dense, colbert]:
            for rank, hit in enumerate(results):
                rrf[hit.id] = rrf.get(hit.id, 0) + 1/(self.k + rank + 1)
        return sorted(rrf.items(), key=lambda x: x[1], reverse=True)[:top_n]

    def crag_grade(self, passages, query):
        graded = [(p, g) for p in passages
                  if (g := self.llm.classify(f"Relevant to '{query}'? {p.text}")) != "Incorrect"]
        if not graded:
            new_q = self.llm.generate(f"Rephrase: {query}")
            return self.search(new_q, self.embedder.encode(new_q), {})
        return graded
```

6.3 Proof-of-Concept: Bilingual Query Pipeline (Arabic + English)

Fatat Al Arab accepts queries in Arabic OR English. The following pseudocode shows the bilingual query intake, translation, and dual-language multi-query expansion:

```
class BilingualQueryPipeline:

    def __init__(self, llm, embedder, colbert, qdrant):
        self.llm = llm
        self.embedder = embedder
        self.colbert = colbert
        self.qdrant = qdrant

    def detect_language(self, query: str) -> str:
        """Returns 'ar', 'en', or 'mixed'"""
        from langdetect import detect
        try:
            lang = detect(query)
            return lang if lang in ['ar', 'en'] else 'mixed'
        except:
            return 'ar'  # Default to Arabic for Nabati corpus

    def translate_to_arabic(self, english_query: str) -> str:
        prompt = (f"Translate this query to Modern Standard Arabic "
                  f"for searching Nabati poetry: '{english_query}'")
        return self.llm.generate(prompt)

    def expand_bilingual(self, query_ar: str, query_en: str) -> list:
        """Generate 3-5 Arabic variants + 3-5 English variants"""
        ar_prompt = (f"Generate 4 Arabic rephrasings of this Nabati poetry "
                     f"search query, including dialect synonyms: {query_ar}")
        en_prompt = (f"Generate 4 English rephrasings of this poetry "
                     f"search query: {query_en}")
        ar_variants = self.llm.generate(ar_prompt).split('\n')
        en_variants = self.llm.generate(en_prompt).split('\n')
        return ar_variants + en_variants  # e.g., for 'longing':
        # AR: [غياب، اشتياق، حنين، وجد]
        # EN: ['separation', 'yearning', 'exile', 'nostalgia']

    def process(self, user_query: str) -> dict:
        lang = self.detect_language(user_query)
        if lang == 'en':
            query_ar = self.translate_to_arabic(user_query)
            query_en = user_query
```

```
elif lang == 'mixed':
    query_ar = user_query  # Arabic part drives retrieval
    query_en = self.llm.generate(f"Extract English part: {user_query}")
else:
    query_ar = user_query
    query_en = self.llm.generate(f"Translate to English: {user_query}")

# Expand in both languages
all_variants = self.expand_bilingual(query_ar, query_en)

# HyDE: hypothetical Nabati verse (always in Arabic)
hyp_verse = self.llm.generate(
    f"Write a short Nabati verse answering: {query_ar}")
hyde_emb = self.embedder.encode(hyp_verse)

# Retrieve on all variants, merge with RRF
all_results = []
for variant in all_variants:
    emb = self.embedder.encode(variant)
    hits = self.qdrant.search(emb, limit=20)
    all_results.append(hits)
# RRF fusion across all variant result lists (k=60)
rrf = {}
for results in all_results:
    for rank, hit in enumerate(results):
        rrf[hit.id] = rrf.get(hit.id, 0) + 1 / (60 + rank + 1)
top5 = sorted(rrf.items(), key=lambda x: x[1], reverse=True)[:5]

return {
    "query_lang": lang,
    "query_ar": query_ar,
    "query_en": query_en,
    "hyde_embedding": hyde_emb,
    "top5": top5,
    # Response will be in the SAME language the user queried in
    "response_lang": lang if lang in ['ar', 'en'] else 'ar'
}
```

6.4 Implementation Impact

These PoCs validate: (1) the 8-challenge preprocessing pipeline is tractable as a deterministic stage sequence; (2) quality triage prevents wasted compute on unrecoverable pages; (3) HyDE + ColBERT + CRAG together address the three main failure modes of poetry RAG (query-poetry embedding gap, dialectal token precision, and synthesis hallucination); and (4) the bilingual pipeline extends the system to international scholars and students who may query in English, while ensuring all retrieval is grounded in the Arabic Khaleeji poetry corpus.

Section 7: Evaluation Strategy

7.1 Al-Nassikh Metrics

Metric	Target	Method
Citation Precision	> 95%	Manual audit of 50 responses; a success is recorded only if the inline citation links directly to the correct manuscript folio.
Hallucination Rate	< 2%	Detection of "out-of-knowledge" generation; flagging any response or verse not present in the local Vector DB.
Response Relevance	> 85%	3-point scale qualitative assessment (Relevant/Partial/Irrelevant) based on alignment with user intent and Nabati context.
Retrieval Recall@5	> 0.85	Testing 30 dialectal queries to ensure the correct manuscript segment appears within the top 5 retrieved results.

7.2 Fatat Al Arab Metrics

Metric	Target	Method
Citation Precision	> 95%	Manual audit of 50 responses; a success is recorded only if the inline citation links directly to the correct manuscript folio.
Hallucination Rate	< 2%	Detection of "out-of-knowledge" generation; flagging any response or verse not present in the local Vector DB.
Response Relevance	> 85%	3-point scale qualitative assessment (Relevant/Partial/Irrelevant) based on alignment with user intent and Nabati context.
Retrieval Recall@5	> 0.85	Testing 30 dialectal queries to ensure the correct manuscript segment appears within the top 5 retrieved results.

Section 8: Risks and Mitigation

Risk	Impact	Prob.	Mitigation
Hallucination in Heritage	Critical	Low	**Strict Grounding :**Mandatory inline citations; LLM is restricted to synthesis using only verified Vector DB text.
Extreme Page Degradation	High	Medium	**Expert Triage :**Manual exclusion of illegible pages during the data selection phase to maintain training set quality.
HTR Accuracy Loss (Bleed/Ink(	High	Medium	**Human-in-the-Loop :**Manual transcription in eScriptorium ignores visual noise, providing "clean" labels for model training.
Column/Layout Merge Errors	High	High	**Manual Segmentation** %100 **:**manual "Scalpel" correction of Sadr/Ajuz baselines to ensure perfect structural alignment.
Annotation/Verse Mixing	Medium	High	**Semantic Labeling :**Using eScriptorium ontology to manually separate marginalia from the primary poetic corpus.
Diacritics Loss	High	High	**Expert Interpretation :**Annotator uses poetic meter and dialectal knowledge to restore missing diacritics during transcription.
Schedule Slippage	High	Medium	**Scope Management :**Prioritizing the 50-page "Gold Standard" set over complex RAG optimizations if time becomes limited.
Vector DB Mismatch	Medium	Low	**Metadata Enrichment :**Manually tagging every entry with Poet, Source, and Folio ID to ensure 100% retrieval traceability.

References

[1] A Deep Learning Approach for Arabic Manuscripts Classification - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC10575097/

[2] HATFormer: Historic Handwritten Arabic Text Line Recognition with Transformers - arXiv, https://arxiv.org/html/2410.02179v2

[3] Arabic Image to Text (OCR) - GitHub, https://github.com/IsmaelMousa/arabic-image-to-text

[4] A Systematic Literature Review of Deep Learning Methods for Handwritten Text Recognition in Historical Arabic Manuscripts, https://etasr.com

[5] QARI-OCR: High-Fidelity Arabic Text Recognition via Multimodal LLM Adaptation - arXiv, https://arxiv.org/html/2506.02295v1

[6] Analysis of Recent Deep Learning Techniques for Arabic Handwritten-Text OCR - MDPI

[7] Paper page - QARI-OCR - Hugging Face, https://huggingface.co/papers/2506.02295

[8] Text Extraction from Historical Documents in Arabic - YouTube

[9] Challenges and Solutions in Developing Accurate Arabic OCR Technology - Accura Scan

[10] Handwriting OCR – AI-Powered Recognition - Transkribus

[11] Designing an Arabic Handwritten Segmentation System - CORE

[12] Deep CNN for isolated Arabic handwritten character recognition - PMC

[13] HICMA: Handwriting Identification for Calligraphy and Manuscripts in Arabic - ACL Anthology

[14] Supporting Undotted Arabic with Pre-trained Language Models - ACL Anthology

[15] A scarce dataset for ancient Arabic handwritten text recognition - PMC

[16] YouTube - Digitizing Arabic Manuscripts at the Bodleian Library, Oxford

[17] Challenges of Layout Analysis across Arabic-Script Training Data

[18] A One-Shot Learning Approach To Document Layout Segmentation of Ancient Arabic Manuscripts - WACV 2024

[19] A Classification of Al-hur Arabic Poetry - IJCSM

[20] A Survey of OCR in Arabic Language: Applications, Techniques, and Challenges - MDPI

[21] Automated compilation of Urdu poetry handwritten image datasets - PMC

[22] Layout Analysis of Arabic Script Documents - Semantic Scholar

[23] I built a deterministic engine to analyse 8th-century Arabic Poetry meters - Reddit

[24] cnemri/pyarud: Arabic Arud (prosody) toolkit for Python - GitHub

[25] Integrating CNN and transformer architectures for Arabic handwriting - PMC

[26] Determining the meter of classical Arabic poetry using deep learning - Frontiers

[27] A Comparative Study of Four HTR Models in Arabic Script - ResearchGate

[28] HATFormer - OpenReview

[29] HATFormer Review - Moonlight

[30] HATFORMER - OpenReview PDF

[31] HATFormer - arXiv (v1)

[32] Arabic NLP Series - Episode 22: Arabic OCR Models - YouTube

[33] NAMAA-Space/Qari-OCR-0.2.2.1-VL-2B-Instruct - Hugging Face

[34] NAMAA-Space/Qari-OCR-v0.3-VL-2B-Instruct - Hugging Face

[35] oddadmix/Katib-Qwen3.5-0.8B-0.1 - Hugging Face

[36] Arabic-Qwen3.5-OCR-v4 - Reddit

[37] YouTube (Arabic) - Best OCR model 2025

[38] We tested every VLM for Arabic document extraction - Reddit r/LocalLLaMA

[39] At the Dawn of Digital Studies on Arabic Script in France - The Digital Orientalist

[40] 01 OPEN HTR WITH ESCRIPTORIUM AND KRAKEN - YouTube

[41] GitHub - mittagessen/kraken: OCR engine for all the languages

[42] Advances and Limitations in Open Source Arabic-Script OCR - hcommons

[43] Advances and Limitations in Open Source Arabic-Script OCR - Digital Studies

[44] Training with eScriptorium - UB Mannheim GitHub

[45] Hands-on Introduction to eScriptorium

[46] Using eScriptorium for Manuscript Transcription - Open Islamicate Texts Initiative

[47] Can AI read the Arabic script? - Transkribus Blog

[48] How to choose the best Transkribus model - Blog

[49] Agapet17 - Arabic Text Recognition Model - Transkribus

[50] Is it worth training a model on Transkribus? - Blog

[51] Transkribus: Reviewing HTR training on manuscripts - RIDE

[52] Character Error Rate and Learning Curve - Transkribus Help Center

[53] Assessing advanced handwritten text recognition engines - ResearchGate

[54] OCR and handwritten text recognition - Calfa OCR

[55] Research Plan - Calfa OCR

[56] Ground-truth of the Tarima project (HTR of Maghrebi Arabic documents) - GitHub

[57] Zinki - Manuscripts, https://zinki.ai/manuscripts-ar/

[58] Launch @GedoLibrary_bot - Telegram

[59] Makhtotatna - Telegram

[60] Leveraging OCR and AI to Explore Arabic Manuscript Collections - YouTube

[61] msfasha/Arabic-Deep-Learning-OCR - GitHub

[62] Arabic OCR.ipynb - Google Colab

[63] Muharaf: Manuscripts of Handwritten Arabic Dataset - arXiv

[64] Muharaf - NeurIPS 2024 Proceedings

[65] A Comparative Study of Four HTR Models in Arabic Script - IIETA

[66] mbzuai-oryx/KITAB-Bench - GitHub

[67] KITAB-Bench: Multi-Domain Benchmark for Arabic OCR - ACL Anthology

[68] Sowayan, S. A. (1985). Nabati Poetry: The Oral Poetry of Arabia. University of California Press.

[69] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 33.

[70] LangChain (2024). LangGraph + SOTA RAG Techniques (Video + Architectural Diagram).

[71] UNESCO (2023). Intangible Cultural Heritage in the UAE.